

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

```
import os
import shutil
from sklearn.model_selection import train_test_split
from google.colab import drive

# Define paths
dataset_folder = "/content/drive/MyDrive/ALzh" # Original dataset folder
output_train_folder = "/content/drive/MyDrive/RESULT/Train"
output_test_folder = "/content/drive/MyDrive/RESULT/Test"

# Ensure output directories exist
os.makedirs(output_train_folder, exist_ok=True)
os.makedirs(output_test_folder, exist_ok=True)

# Define labels based on subfolder names
labels = ['AD updated', 'CN Updated', 'MCI Updated']

# Check and process each category folder
for label in labels:
    section_path = os.path.join(dataset_folder, label)

    # Skip if not a directory
    if not os.path.isdir(section_path):
        print(f"Directory not found for label: {label}. Skipping...")
        continue

    # List all files in the section folder
    files = [f for f in os.listdir(section_path) if os.path.isfile(os.path.join(section_path, f))]
    print(files)
    if not files:
        print(f"No files found in section: {label}. Skipping...")
        continue

    # Split files into training and testing sets
    train_files, test_files = train_test_split(files, test_size=0.2, random_state=42)

    # Create output directories for the current section
    train_section_path = os.path.join(output_train_folder, label)
    test_section_path = os.path.join(output_test_folder, label)
    os.makedirs(train_section_path, exist_ok=True)
    os.makedirs(test_section_path, exist_ok=True)

    # Copy training files
    for file in train_files:
        src = os.path.join(section_path, file)
        dst = os.path.join(train_section_path, file)
        shutil.copy(src, dst)

    # Copy testing files
    for file in test_files:
        src = os.path.join(section_path, file)
        dst = os.path.join(test_section_path, file)
        shutil.copy(src, dst)

    # Print summary
    total_files = len(files)
    train_count = len(train_files)
    test_count = len(test_files)
    print(f"Section: {label}")
    print(f"Total files: {total_files}")
    print(f"Training files: {train_count} ({(train_count / total_files) * 100:.2f}%)")
    print(f"Testing files: {test_count} ({(test_count / total_files) * 100:.2f}%)")
    print("-" * 50)

print("Dataset successfully split into training and testing sets!")
```

```
↳ ['ADNI_002_S_1018_MR_MPR____N3__Scaled_Br_20070217032215330_S24312_I40828_sagittal_slice_116.jpg', 'ADNI_002_S_1018_MR_MPR____N3__Sc
```

```
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-2-dfeebc873ca9> in <cell line: 0>()
    45         src = os.path.join(section_path, file)
    46         dst = os.path.join(train_section_path, file)
--> 47         shutil.copy(src, dst)
    48
    49         # Copy testing files
```

```
┆ 1 frames ┆
/usr/lib/python3.11/shutil.py in copyfile(src, dst, follow_symlinks)
    256         with open(src, 'rb') as fsrc:
    257             try:
--> 258                 with open(dst, 'wb') as fdst:
    259                     # macOS
    260                     if _HAS_FCOPYFILE:
```

```
KeyboardInterrupt:
```

```
import cv2
import numpy as np
from tqdm import tqdm
from sklearn.utils import shuffle
from tensorflow.keras.utils import to_categorical

# Paths for train and test folders
train_path = "/content/drive/MyDrive/RESULT/Train"
test_path = "/content/drive/MyDrive/RESULT/Test"

# Labels corresponding to folder names
labels = ['AD updated', 'CN Updated', 'MCI Updated']

# Image size for resizing
image_size = 150

# Helper function to load and preprocess images
def load_images(folder_path, label, data_list, label_list):
    for image_file in tqdm(os.listdir(folder_path), desc=f"Loading {label}"):
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path) # Load the image
        if image is not None:
            image = cv2.resize(image, (image_size, image_size)) # Resize to 150x150
            data_list.append(image)
            label_list.append(label)

# Initialize lists for training and testing data
X_train, Y_train = [], []
X_test, Y_test = [], []

# Load training data
for label in labels:
    folder_path = os.path.join(train_path, label)
    if os.path.exists(folder_path):
        load_images(folder_path, label, X_train, Y_train)
    else:
        print(f"Training folder for {label} not found. Skipping...")

# Load testing data
for label in labels:
    folder_path = os.path.join(test_path, label)
    if os.path.exists(folder_path):
        load_images(folder_path, label, X_test, Y_test)
    else:
        print(f"Testing folder for {label} not found. Skipping...")

# Convert to NumPy arrays
X_train = np.array(X_train)
Y_train = np.array(Y_train)
X_test = np.array(X_test)
Y_test = np.array(Y_test)

# Shuffle training data
X_train, Y_train = shuffle(X_train, Y_train, random_state=42)

# Convert labels from strings to integers
label_to_int = {label: idx for idx, label in enumerate(labels)}
Y_train_int = np.array([label_to_int[label] for label in Y_train])
Y_test_int = np.array([label_to_int[label] for label in Y_test])
```

```

# One-hot encode the labels
Y_train_onehot = to_categorical(Y_train_int, num_classes=len(labels))
Y_test_onehot = to_categorical(Y_test_int, num_classes=len(labels))

# Print shapes for verification
print("Training data shape:", X_train.shape)
print("Training labels shape:", Y_train_onehot.shape)
print("Testing data shape:", X_test.shape)
print("Testing labels shape:", Y_test_onehot.shape)

Loading AD updated: 100%|██████████| 427/427 [00:04<00:00, 95.36it/s]
Loading CN Updated: 100%|██████████| 730/730 [00:08<00:00, 86.12it/s]
Loading MCI Updated: 100%|██████████| 715/715 [00:07<00:00, 90.50it/s]
Loading AD updated: 100%|██████████| 107/107 [00:00<00:00, 120.97it/s]
Loading CN Updated: 100%|██████████| 183/183 [00:01<00:00, 108.13it/s]
Loading MCI Updated: 100%|██████████| 179/179 [00:01<00:00, 112.88it/s]
Training data shape: (1872, 150, 150, 3)
Training labels shape: (1872, 3)
Testing data shape: (469, 150, 150, 3)
Testing labels shape: (469, 3)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Define CNN model
model1 = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(image_size, image_size, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(3, activation='softmax') # 3 classes: AD, CN, MCI
])

# Compile the model
model1.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'] # Only accuracy is used as a metric
)

# Train the CNN model
history1 = model1.fit(
    X_train, Y_train_onehot,
    epochs=15, # Number of epochs
    batch_size=32, # Batch size
    verbose=1
)

# Evaluate the model on the test set
test_loss1, test_accuracy1 = model1.evaluate(X_test, Y_test_onehot, verbose=2)
print(f"Test Loss: {test_loss1:.4f}")
print(f"Test Accuracy: {test_accuracy1 * 100:.2f}%")

# Predictions on the test set
pred1 = model1.predict(X_test)
pred1 = np.argmax(pred1, axis=1)
actual_label1 = np.argmax(Y_test_onehot, axis=1)

# Classification report
labels_final = ['AD', 'CN', 'MCI']
print(classification_report(actual_label1, pred1, target_names=labels_final))

# Confusion matrix
cm1 = confusion_matrix(actual_label1, pred1)
plt.figure(figsize=(8, 6))
sns.heatmap(cm1, cmap='Blues', annot=True, fmt='d', xticklabels=labels_final, yticklabels=labels_final)
plt.title("Confusion Matrix for Alzheimer's Disease Classification (CNN Model)")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.savefig('cnn_confusion.png', dpi=300)

```

plt.show()

```

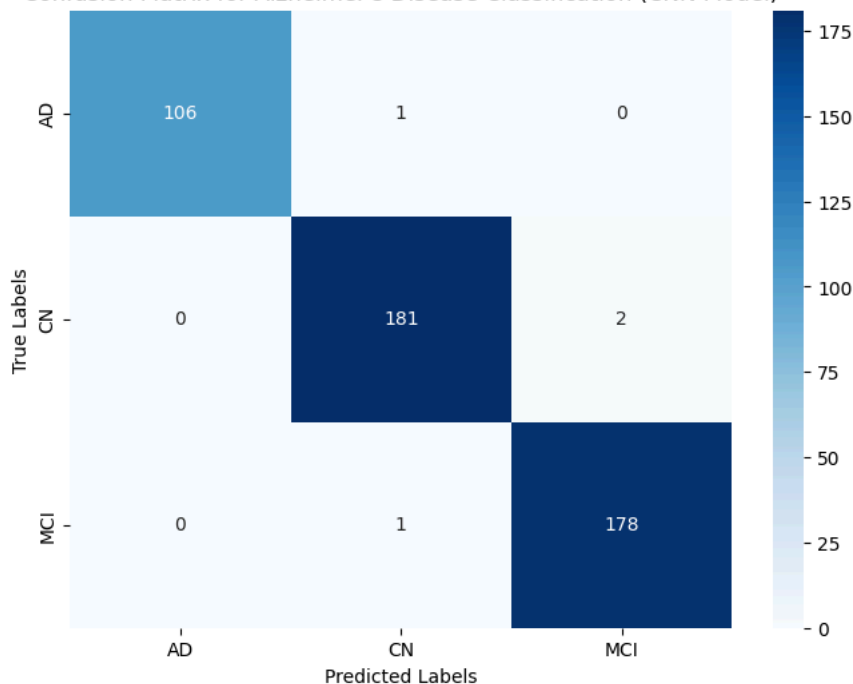
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape` to
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/15
59/59 ━━━━━━━━━━━ 90s 1s/step - accuracy: 0.4165 - loss: 33.7062
Epoch 2/15
59/59 ━━━━━━━━━━━ 136s 1s/step - accuracy: 0.6718 - loss: 0.7464
Epoch 3/15
59/59 ━━━━━━━━━━━ 83s 1s/step - accuracy: 0.8335 - loss: 0.4221
Epoch 4/15
59/59 ━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9179 - loss: 0.2305
Epoch 5/15
59/59 ━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9570 - loss: 0.1270
Epoch 6/15
59/59 ━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9640 - loss: 0.0950
Epoch 7/15
59/59 ━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9644 - loss: 0.0970
Epoch 8/15
59/59 ━━━━━━━━━━━ 83s 1s/step - accuracy: 0.9833 - loss: 0.0509
Epoch 9/15
59/59 ━━━━━━━━━━━ 141s 1s/step - accuracy: 0.9848 - loss: 0.0463
Epoch 10/15
59/59 ━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9902 - loss: 0.0376
Epoch 11/15
59/59 ━━━━━━━━━━━ 141s 1s/step - accuracy: 0.9819 - loss: 0.0454
Epoch 12/15
59/59 ━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9886 - loss: 0.0475
Epoch 13/15
59/59 ━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9844 - loss: 0.0515
Epoch 14/15
59/59 ━━━━━━━━━━━ 81s 1s/step - accuracy: 0.9855 - loss: 0.0438
Epoch 15/15
59/59 ━━━━━━━━━━━ 82s 1s/step - accuracy: 0.9857 - loss: 0.0410
15/15 - 5s - 344ms/step - accuracy: 0.9915 - loss: 0.0236
Test Loss: 0.0236
Test Accuracy: 99.15%
15/15 ━━━━━━━━━━━ 8s 502ms/step
          precision    recall  f1-score   support

     AD         1.00        0.99         1.00        107
     CN         0.99        0.99         0.99        183
     MCI         0.99        0.99         0.99        179

 accuracy          0.99          0.99          0.99          469
 macro avg         0.99          0.99          0.99          469
 weighted avg         0.99          0.99          0.99          469

```

Confusion Matrix for Alzheimer's Disease Classification (CNN Model)

Start coding or [generate](#) with AI.